Expert Systems With Applications 313 (2026) 131567



Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa



Quantformer: From attention to profit with a quantitative transformer trading strategy

Zhaofeng Zhang [a,b,c], Banghao Chen [a,d,e], Shengxin Zhu [f,a,d,*], Nicolas Langrené [a,b,*]

[a] Guangdong Provincial/Zhuhai Key Laboratory of Interdisciplinary Research and Application for Data Science, Beijing Normal-Hong Kong Baptist University, Zhuhai, Guangdong, 519087, China
[b] Department of Mathematical Sciences, Beijing Normal-Hong Kong Baptist University, Zhuhai, Guangdong, 519087, China
[c] Department of Mathematics, University of Michigan, Ann Arbor, Michigan, 48109, United States
[d] Department of Statistics and Data Science, Beijing Normal-Hong Kong Baptist University, Zhuhai, Guangdong, 519087, China
[e] Department of Biomedical Informatics, Yong Loo Lin School of Medicine, National University of Singapore, Singapore, 119228, Singapore
[f] Advanced Institute of Natural Science, Beijing Normal University, Zhuhai, Guangdong, 519087, China

ARTICLE INFO

Keywords:
Quantformer
Transformer
Neural networks
Quantitative finance
Stock selection
Portfolio optimization
Market sentiment

ABSTRACT

In traditional quantitative trading practice, navigating the complicated and dynamic financial market presents a persistent challenge. Fully capturing various market variables, including long-term information, as well as essential signals that may lead to profit remains a difficult task for learning algorithms. In order to tackle this challenge, this paper introduces quantformer, an enhanced neural network architecture based on transformer, to build investment factors. By borrowing ideas from sentiment analysis, quantformer not only exploits its original inherent advantages in capturing long-range dependencies and modeling complex data relationships, but is also able to solve tasks with numerical inputs and accurately forecast future returns over a given period. This work collects more than 5,000,000 rolling data of 4601 stocks in the Chinese capital market from 2010 to 2023. The results of this study demonstrate the model's superior performance in predicting stock trends compared with other 100-factor-based quantitative strategies. Notably, the model's innovative use of transformer-like model to establish factors, in conjunction with market sentiment information, has been shown to enhance the accuracy of trading signals significantly, thereby offering promising implications for the future of quantitative trading strategies. The implementation details and code is available on Github.

1. Introduction

The goal of stock trading is to optimize the return on investment in the capital market according to the process of buying or selling one or more companies' shares. Traders obtain profit when a positive difference is generated by the fluctuation of stock price. However, stocks are influenced by a large number of factors, which constitute a complex system and make it difficult for people to make a profit. The assessment of a stock's evolving trend is inherently challenging due to the highly volatile and interconnected nature of the market, which sets it apart from typical time series modeling (Wang et al., 2022b). As a result, many strategies and tools have been built, in parallel with the development of capital markets, and quantitative strategies have been playing an important role among them.

Some traditional quantitative tools, such as the Markowitz portfolio theory (Markowitz, 1952) and the Capital Asset Pricing Model (CAPM) (Sharpe, 1964), focus mainly on static fundamental analysis. In other words, these strategies aim to make a profit by simple calculation and analysis. Since then, along with the development of computer science, more quantitative methods and tools have been introduced. Within these methods, factor-based strategies have attracted much attention. In 1993, Fama and French (1993) introduced the Fama-French Three Factor Model (FF3), which has become an influential model in quantitative trading. In 2015, Fama and French revised their model with a Five-Factor Asset Pricing Model (FF5) (Fama & French, 2015). Besides this classical theory, numerous trading strategies have been published for decades. For example, Shi et al. (2024) built an optimal portfolio to catch Future Investment Opportunities (FIO) by multi-factor models.

Quantitative trading with factors typically follows two primary approaches, which are shown in Fig. 1. The first approach involves the computation of stock factor values. Based on these calculated values,

* Corresponding authors.
E-mail addresses: zhangzf@umich.edu (Z. Zhang), chenbanghao@u.nus.edu (B. Chen), shengxin.zhu@bnu.edu.cn (S. Zhu), nicolaslangrene@bnbu.edu.cn (N. Langrené).

https://doi.org/10.1016/j.eswa.2026.131567
Received 18 August 2025; Received in revised form 22 December 2025; Accepted 2 February 2026
Available online 10 February 2026
0957-4174/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

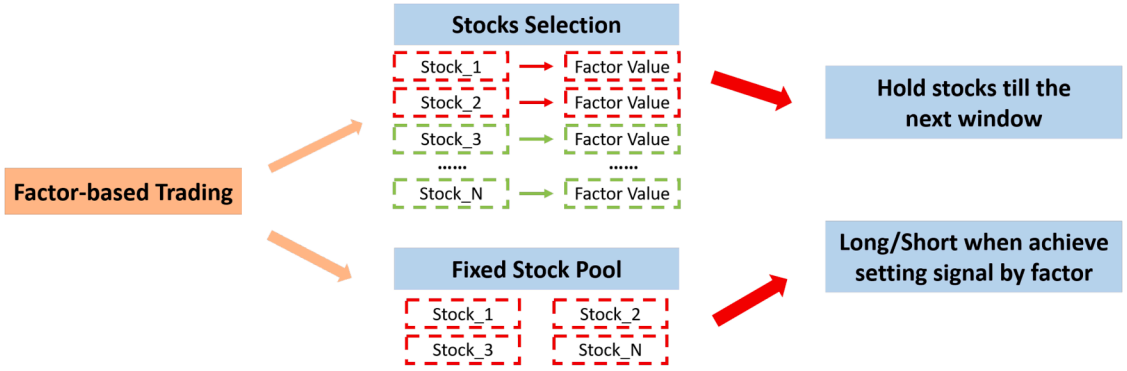


Fig. 1. Quantitative trading architecture with factors.

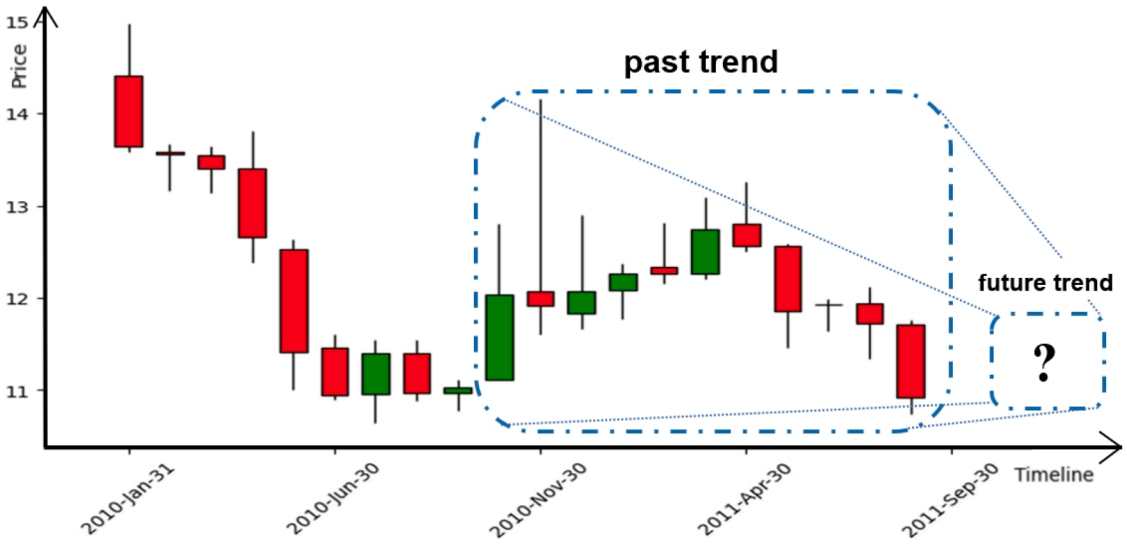


Fig. 2. Example of stock prediction.

stocks are ranked to establish a pool. Once this pool is established, assets are held for a predetermined period. Adjustments to the portfolio are then made at specific time intervals, ensuring alignment with evolving market conditions and factor readings. The second method employs a fixed pool of stocks, wherein factors guide the derivation of long/short signals. Traders can execute corresponding actions when they receive the signals from factors, allowing for a dynamic response to market fluctuations based on factor insights. In recent years, Machine Learning (ML) has become an instrumental tool in trading algorithms and decision-making processes. ML methods allow systems to learn from and make decisions based on data, and lend themselves particularly well to the vast and dynamic landscapes of stock markets. For example, Nayak et al. (2015) exploited machine learning algorithms based on manual indicators, but the underlying random walk hypothesis may hamper the task of understanding inherently non-stationary series. With the rise of different architectures, Feng et al. (2019) introduced Relational Stock Ranking (RSR) to catch the sentiment of the market in trading data. Kehinde et al. (2025b) propose a model about LOESS (STL) and Extreme Learning Machine (ELM) for high-frequency financial forecasting. However, modern ML models also face challenges in real-world trading environments, such as the limitation in their ability to simultaneously achieve predictive accuracy, computational efficiency, and stable trading performance (Kehinde et al., 2023).

Although there exist several previous experiments with ML factors that attempted to fetch market sentiment in the quantitative finance field, this research still faces two difficulties. Firstly, in the field of sentiment analysis, which is a branch of Natural Language Processing (NLP, a field of computer science that aims to understand, interpret, and generate human language), models are used to convert words in text to word vectors through word embeddings to serve as inputs. However, instead of words, financial datasets contain both categorical data, such as industry types, as well as quantitative data, such as price fluctuation, turnover rate, and financial indicators. If the input comprises only categorical data, the time series can be treated as a sentence (Gorishniy et al., 2022). In most cases, the input will involve numerical data, which cannot be transformed via word embeddings. However, in many quantitative financial applications, the input consists of continuous numerical variables, which are not naturally represented by word embeddings designed for discrete tokens.

Secondly, most NLP tasks can be transformed into sequence-to-sequence (seq2seq) problems, such as in machine translation, dialogue systems, and speech recognition. As an example, the transformer architecture is based on the seq2seq architecture (Vaswani et al., 2017). To utilize existing outputs, decoders in transformer sequentially output samples and use masking operations to handle input sequences during training. However, in stock prediction, where the aim is often to accurately forecast future returns over a period, as shown in Fig. 2, the transformer model is rarely used for such tasks.

To address these problems, we propose *quantformer*, a modified transformer architecture adapted to rolling quantitative data and quantitative financial tasks, and use it as an investment factor. Quantformer is able to input numerical data directly, which refers to a method similar to sentiment analysis. Our contributions lie in the following aspects:

- We propose quantformer, whose structure adapts to rolling stock-related time series data as inputs without positioning module nor word embedding. The new structure with linear embedding shows better fitting to the numerical type inputs.
- According to our experiments, quantformer-based factors perform better in back-tests compared with other 100 traditional factor strategies under the same time period. Factors are evaluated by various metrics, such as annualized return, Sharpe Ratio and Value at Risk (VaR).
- We experiment the performance of quantformer with different trading frequencies (daily, weekly, and monthly) and labeling schemes (20%, 10%, 5% and 1% scale of the stocks in the market). These experiments indicate a robust potential direction for applying transformer-based models to quantitative financial tasks.

The paper is structured as follows. Section 2 discusses previous quantitative financial works based on machine learning. Sections 3 and 4 introduce quantformers. A factor based on quantformer will be trained and back-tested. For the practical backtest, we collected data from more than 4600 stocks in the past 14 years (from 2010 to 2023). To comprehensively test the ability of the factor, we divide the data by different frequencies (Section 4.2) and train under different training scales (Section 5.2). Finally, the results of the back-tests including the comparison between the quantformer-factor and other 100 factors as well as the insights gained from such comparative analysis are discussed in Sections 4 and 5.

2. Preliminaries

This section briefly introduces the related works about stock prediction with market sentiment and the development of quantitative financial trading methods, especially those based on Machine Learning (ML) methods.

2.1. Stock data and market sentiment

It is known that, in some cases, stock data can be deemed to influence the future trend of a stock price (Asness, 1995). Chen et al. (2020), Haritha and Rishad (2020) have discussed the correlation between investor sentiment and stock market metrics such as turnover rate, indicating that investor feelings, both individual and institutional, significantly influence market behavior, affecting aspects such as volatility and returns. This relationship is not uniform but varies with changes in investor sentiment. It also highlights that investor psychology, especially during periods of high market stress can drastically impact market behavior and investor decisions. This link between sentiment and market performance is further affirmed by studies focusing on factors such as the turnover ratio, which are seen as reflective of market liquidity and investor behavior (Naseem et al., 2021). Ding et al. (2023) built a new factor based on sentiment analysis with the average of trading signals from technical trading strategies to benchmark stocks of the S&P 500 index and DJIA. The sentiment factor shows correlation with stock returns. These findings underscore the mutual relationship between investor sentiment and market performance, demonstrating how psychological factors can drive market dynamics.

2.2. Applications of machine learning methods for trading

Machine learning exploits a range of algorithms and statistical techniques used for tasks such as regression, clustering, and classification. In past decades, there have been methods that have been increasingly applied to stock data due to their ability to process vast amounts of data and make predictions based on them, offering a potential advantage in the financial markets. At the same time, the financial sector, particularly quantitative trading, has started to be aware of the potential of deep learning models for predicting stock movements, portfolio optimization, and risks.

2.2.1. Support vector machines (SVMs)

SVMs have been used in quantitative trading, as they can manage high-dimensional spaces and intricacies found in datasets. Kim (2003) used SVMs to forecast financial time series in 2003, with a focus on the South Korean stock market, exhibiting the robustness of SVM against traditional statistical methods and some other computational techniques. Similarly, Huang et al. (2005) showcased SVMs' prowess in anticipating the movement direction of Japanese stock markets, especially when determining bullish or bearish market sentiments. Cavalcante et al. (2016) assessed SVMs' adaptability and precision in financial tasks such as portfolio and quantitative trading.

However, SVMs can also show some disadvantages in quantitative trading. In Huck (2009)'s work, SVM was underscored for the accuracy of overfitting. When applied to financial datasets, the model's propensity to fit too closely to the training data might lead to reduced generalization capabilities, making the strategy ineffective on unseen data. In high-frequency trading, though the model displayed competency in specific scenarios, researchers pinpointed potential limitations when grappling with certain volatile market dynamics, suggesting the need for adaptations (Kercheval & Zhang, 2015).

2.2.2. Long short-term memory (LSTM)

LSTM is a neural network architecture designed to capture long-term dependencies in sequential data (Hochreiter & Schmidhuber, 1997). LSTM networks can accentuate and effectively model volatile financial markets, showcasing significant improvements over traditional time series models. Orsel and Yamada (2022) combined LSTM with linear Kalman filter and set experiments on different volatility stocks to build strategies. Bao et al. (2017) deployed LSTM with stacked autoencoders to forecast financial time series. Fischer and Krauss (2018a) tested the efficacy of LSTM for price predictions. They observed LSTMs' prowess in noisy data and their superiority in prediction accuracy over other conventional models. Sezer and Ozbayoglu (2018) combined LSTM with other architectures and found that these can enhance the feature extraction process, thereby improving prediction accuracy. For predicting stock prices, Zhang et al. (2017) harnessed the power of LSTM to unearth multi-frequency trading patterns. Their methods show the adaptability of LSTMs in modeling complex situations inherent in stock prices.

2.2.3. Gated recurrent units (GRUs)

GRUs (Chung et al., 2014) are another neural network architecture which has emerged as a noteworthy method in the quest for accurate financial forecasting. GRUs can capture long-term dependencies in time series data and have seen increasing popularity in the world of quantitative trading. Bao et al. (2017) introduced a comprehensive deep-learning framework for financial time series, which incorporated GRUs alongside LSTM and stacked autoencoders. Their model, when applied to diverse datasets, performs better than traditional time series models, accentuating the adaptability and robustness of GRUs. Fischer and Krauss (2018b) centralized on LSTM networks, provided comparative insights into GRUs. However, in their analysis, LSTMs showed a slight edge in prediction accuracy, indicating that while GRUs are powerful, selecting between them and LSTMs may boil down to specific use cases and computational constraints. Beyond that, Pak et al. (2018) presented a model fusing GRUs with Convolutional Neural Networks (CNN) O'Shea and Nash (2015). The strategy to combine CNNs and GRUs also exhibits advantages for stock price prediction. Sun et al. (2024) introduced the Attention-GRU model which combined attention and GRU to establish a new factor based on CVaR portfolio (Ban et al., 2018; Zhu & Fukushima, 2009). The Attention-GRU model fitted market return, stocks return from 28 Dow Jones Industrial Average index (DJIA) stocks, which achieved better performance than other models in eight metrics such as annual return, standard deviation and information ratio.

Table 1
Advanced transformer-based architectures for time series data.

Reference	Model	Method
Wu et al. (2020)	Adversarial Sparse Transformer (AST)	Sparse attention with adversarial training
Liang et al. (2023)	AirFormer	Dartboard spatial self-attention and causal temporal self-attention for different training stage
Wang et al. (2022b)	ALSP-TF	Local interaction and global interaction between layers
Wu et al. (2021b)	Autoformer	Auto-correlation instead of attention
Wang (2023)	BiLSTM-MTRAN-TCN	Combined BiLSTM, Transformer and Temporary Revolution Network (TCN)
Kim et al. (2024)	ConvLSTM	Combined LSTM with transformer
Li et al. (2019)	ConvTrans	
Zeng et al. (2023a)	LTSF-Linear	DLinear and NLinear, MLP-based with transformer
So et al. (2019)	Evolved Transformer	Neural Architecture Search (NAS) in transformer
Zhou et al. (2022)	FEDformer	Timestamp encoding with frequency enhanced attention
Liu et al. (2021)	Gated Transformer Networks (GTN)	Two-tower transformer with channel-wise & step-wise
Ding et al. (2020)	Gaussian transformer	Multi-scale Gaussian prior, orthogonal regularization and trading gap splitter for self-attention
Barez et al. (2023)	HFformer	Linear decoder and spiking activation
Kehinde et al. (2025a)	Helformer	
Zhou et al. (2021)	Informer	ProbSparse self-attention, self-attention distilling operation and generative style decoder
Liu et al. (2024)	iTransformer	Dimension inversion and deleted positioning
Li et al. (2019)	LogSparse Transformer	Causal convolution attention and query-key matching
Eisenach et al. (2020)	MQ-Transformer	Positional encoding from event indicators and decoder-encoder attention for context-alignment
Ramos-Pérez et al. (2021)	Multi-Transformer	Average multi-head and bagging attention and positioning based on time stamps
Zerveas et al. (2021)	Multivariate transformer	Deleted decoder and applied learnable positioning module for multivariate time series
Liu et al. (2022b)	Non-stationary Transformer	Series standardization and De-stationary attention
Nie et al. (2022)	Patch Time Series Transformer (PatchTST)	Divided time series into patches, applied channel-independence and instance normalization
Tang and Matteson (2021)	ProTran	Modeled non-Markovian dynamics by State Space Models (SSMs)
Liu et al. (2022a)	Pyraformer	Pyramidal Attention Module (PAM) and intra-scale neighboring connections model
Chen et al. (2022)	Quatformer	Learning-to-rotate Attention (LRA) with global memory and trend normalization
Kitaev et al. (2020)	Reformer	Locality-sensitive hashing attention and reversible residual layers
Moreno-Pino et al. (2024)	Rough Transformer	Multi-view signature attention and continuous-time paths encoding transformation
Shabani et al. (2022)	Scaleformer	Multi-scale framework with various models
Lin et al. (2021)	State Space Decomposition Neural Network (SSDNet)	Transformer with SSMs
Ma et al. (2025)	Stockformer	Dual-frequency spatiotemporal encoder for data in different frequencies with graph embedding and fusion attention
Chowdhury et al. (2022)	TARNet	Attention score for timestamps masking
Olorunnimbe and Viktor (2024)	Temporal Fusion Transformer (TFT)	LSTM learning positional encoding
Fischer et al. (2024)	Transformer for FX-Spot	Time2Vec positioning module
Zuo et al. (2020)	Transformer Hawkes process (THP)	Positional encoding by translating time intervals
Cirstea et al. (2022)	Triformer	Triangular, variable-specific patch attention

2.3. Transformer in time series prediction

Transformer (Vaswani et al., 2017), which was introduced in 2017, has received considerable attention in the NLP field. According to its special encoder and decoder stacks with self-attention blocks, the transformer shows its advantage in stability, speed, and long-term memory compared with other traditional models. Transformer-based models such as GPT (Radford et al., 2019) and BERT (Devlin et al., 2019) have been influential in the NLP field. More recent transformer-based models include ChatGPT (Brown et al., 2020), FinBERT (Araci, 2019; Yang et al., 2020) and BloombergGPT (Wu et al., 2023).

Similar to other neural sequence transduction models, transformer also has an encoder-decoder structure. The model takes as input a sequence of context, which could be natural language sentences, and generates output with texts depending on the input, similar to a translation according to the input or a predicted sentence. Transformer consists of stacks of layers, which include self-attention and point-wise operations. These layers are used to connect the encoder and decoder in the architecture. For these encoder and decoder layers, the self-attention method plays an important role in matrix calculation. The linear and softmax functions are used in the dot-product attention module. In the vanilla transformer model, multi-head attention layers are used and the result from each layer is outputted after going through an additional normalization layer. The above-mentioned steps cover most of the progress made with the vanilla transformer.

Table 1 shows a set of advanced transformer models, with a focus on time series prediction tasks, which are tasks of interest in quantitative finance. The table highlights the modifications made to the original transformer model. These modifications can be categorized into innovations in attention mechanisms, encoding and decoding structure, positioning module, and mixture model.

Some research proposed enhancements of the attention module. For example, Adversarial Sparse Transformer (AST) (Wu et al., 2020) introduced sparse attention using the α-Entmax function within adversarial training. Autoformer (Wu et al., 2021b) replaced traditional self-attention with an auto-correlation mechanism that utilizes periodic patterns in frequency-domain time series analysis. Similarly, Informer (Zhou et al., 2021) suggested ProbSparse attention that selects only the most relevant queries and introduces generative-style decoding to handle long-term dependencies. Gated Transformer Networks (GTN) (Liu et al., 2021) used separate attention for channel-wise and temporal-wise interactions for multivariate time series. Patch Time Series Transformer

(PatchTST) (Nie et al., 2022) divided time series into patches to model local dependencies, and Quatformer (Chen et al., 2022) attempted to use rotation-based attention to capture periodic patterns in time series. AirFormer (Liang et al., 2023) introduced dartboard spatial attention and causal temporal attention in air quality prediction. Rough transformer (Moreno-Pino et al., 2024) facilitated multi-view signature attention and continuous-time path encoding for financial market modeling for complex financial time series prediction.

In another direction, some works tried to make changes to the encoding or decoding of transformer. FEDformer (Zhou et al., 2022) split time series data into trend and seasonal components with frequency-enhanced modules. To improve computational efficiency, some models, including Multivariate Transformer (Zerveas et al., 2021) gave up using a decoder and focused solely on an encoder for feature extraction for forecasting tasks.

As a kind of revision sight, some research has proposed to replace or extend the positioning modules to better accommodate time series data. For instance, HFformer (Barez et al., 2023) gave up positioning modules, and uses activation functions with a lightweight linear decoder to revise the model. Noting that temporal order is inherently embedded in time series data, iTransformer (Liu et al., 2024) introduced a dimension-inverted structure and removes positioning. Transformer Hawkes Process (THP) (Zuo et al., 2020) put time intervals into the positioning module directly. FX-Spot Transformer (Fischer et al., 2024) used the Time to Vector (Time2Vec) function to handle irregular time intervals.

To overcome the limitations of standalone transformer models, an approach is to integrate them with other frameworks. ConvLSTM (Kim et al., 2024) merged convolutional neural networks (CNNs) and LSTM layers with transformer modules to simultaneously model spatial and temporal patterns. ProTran (Tang & Matteson, 2021) integrated state-space models (SSMs) with transformers to address non-Markovian dynamics in time series data, providing robust long-term forecasts. These hybrid approaches improved the versatility and domain adaptability of transformer-based architectures.

Despite showing some progress in enhancing the original model, many models share several common limitations for time series prediction. First, many models face the problem of high computational and memory costs. This problem limits their scalability to large datasets. This is particularly serious in financial applications, where backtesting over long time periods is often required. Second, some of the models face a robustness problem when processing numerical financial data. Their performance can be sensitive to the choice of training data, its preprocessing, and hyperparameter choices. This influences their robustness and increases implementation complexity in practical quantitative trading. Most models primarily evaluate predictive accuracy using standard forecasting metrics but few of them assess the economic value of the model.

2.4. Transformer in quantitative trading

In the quantitative trading domain, a few research articles have investigated the potential of transformers. Wang et al. (2022a) utilized transformer to predict stock market indices (including CSI 300, S&P 500, Hang Seng Index, and Nikkei 225) and concluded that the model can better catch the rules of stock market dynamics. Ding et al. (2020) exploited transformer on trading sequences to classify stock price movements. Besides using a traditional transformer, other innovative methods of transformer are worth mentioning. Zhou et al. (2021) optimized the efficiency of time complexity and memory usage of transformer on extremely long time series by informer. Zeng et al. (2023b) combined the advantages of CNNs and transformers to model short-term and long-term dependencies in financial time series. They illustrate the merits of this approach on intraday stock price prediction of S&P 500 constituents. ConvLSTM (Kim et al., 2024) has been trained on stocks from the S&P 500 between 2004 and 2021, and performs better than other models such as VAR and ARIMA.

Similar to previous work, Wang et al. (2022b) introduced Adaptive Long-Short Pattern Transformer (ALSP-TF). Their model is structurally designed for hierarchical representation and interaction of stock price series at different context scales. With the help of a learnable function, they make self-attention aware of the weighted time intervals between patterns, to adaptively adjust their dependencies beyond similarity matching. In the end, they obtained more than 10% of annual return on average.

However, the transformer model can also have some disadvantages in practice. Xu et al. (2021) mentioned that the global self-attention module focuses on point-wise token similarities without contextual insights. As fluctuations of stocks are conditioned on composite signals over manifold periods, lacking pattern-wise interaction hinders the adequate discrimination of stock tendency and is susceptible to noise points. On the other hand, Wu et al. (2021a) claimed that the basic query-key matching paradigm is position agnostic. Although position embedding is inserted into the sequential inputs, it may not be optimal because of the inability to reveal precise distances.

3. Methodology

Several researchers have tried to improve transformers for time series and quantitative financial applications and proposed improved models (see time series papers in Table 1), but there are still some aspects that have not been considered. For data in time series type, such as stock prices or daily returns, it is reasonable to assume that each data in the series is assigned a “position" automatically, which is different from sentences. Based on this observation, the positional encoding module on the vanilla model may not be a necessary part. Furthermore, market data are related to the market trend. For example, the return of the stock may include the market sentiment and the turnover rate reflects investor sentiment and trading enthusiasm. Therefore, integrating such market-related features as input can enhance the model’s ability to capture underlying market dynamics and improve the accuracy of future trend predictions. To implement these ideas, we propose *quantformer*, an enhanced transformer-based model for quantitative financial trading, and assess its performance on real-world datasets.

This section introduces the framework of the work and the steps of the establishment of the model, including data processing, quantformer construction and prediction.

3.1. Framework overview

Fig. 3 gives an overview of the structure. There are three major parts:
(i) *Data Initialization* to align the time series of stocks into a regular matrix;
(ii) Unlike the transformer, *embedding* the training dataset and *training* them by a quantformer with word-embedding layer replaced by linear layer and with no Mask(opt.) layer;
(iii) Lastly, using the trained model to *predict* the possible trend of stocks.

3.2. Problem formulation

Training input

Consider a candidate stock set S^t containing N stocks on the trading timestamp t:

$$S^t = \{s_n^t\}_{n=1}^N \tag{1}$$

For a stock s_n^t in the stock set S^t, where $n \in \{1, \dots, N\}$, consider the two-dimensional historical 1-time step (such as one month) input feature matrix $\mathcal{X}_n^t \in \mathbb{R}^{20\times 2}$, consisting of 20 continuous timestamps:

$$\mathcal{X}_n^t = \{x_n^{t-m} | m = 19, 18, \dots, 0\} \tag{2}$$

Each row vector x_n^{t-m} for $m = 0, \dots, 19$ contains two features. The first one is the accumulated daily profit rate r_n^{t-m} during the time step, where

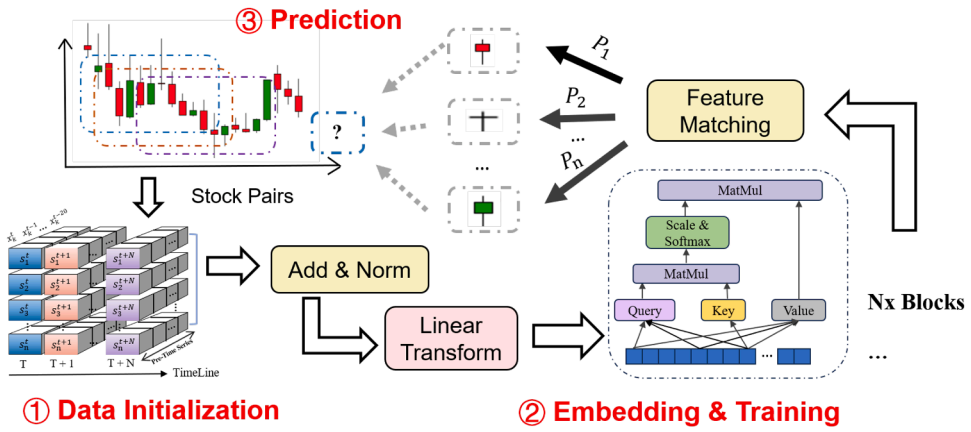


Fig. 3. Overview of the work.

p_n^{t-m} is the close price of the last day at the timestamp $t-m$ and p_n^{t-m-1} is the close price of the last day at the timestamp $t-m-1$.

$$r_n^{t-m} = \frac{p_n^{t-m} - p_n^{t-m-1}}{p_n^{t-m-1}}, \quad v_n^{t-m} = \sum v_n^{t-m,i} \tag{3}$$

The second feature is the accumulated daily turnover rate v_n^{t-m} during the time step $t-m$ for stock n. $v_n^{t-m,i}$ is the daily turnover rate and i is the day index in the time period. To sum up, each $\mathcal{X}_n^t$ can be represented as:

$$\mathcal{X}_n^t = \begin{bmatrix} x_n^{t-19} & x_n^{t-18} & \dots & x_n^t \end{bmatrix}^T \in \mathbb{R}^{20\times 2}, x_n^{t-m} = [r_n^{t-m}, v_n^{t-m}] \tag{4}$$

Normalization

The next step is to normalize the input data for each time step by zero-mean, unit-variance normalization (Z-score). The normalization equation is shown in Eq. (5). For the row vector x_n^t (Eq. (2)), $\mathbb{E}[x^t]$ and $\text{std}[x^t]$ represent the mean and standard deviation of all the row vectors x^t at time t, respectively, and the resulting values $\tilde{x}_n^t$ represent the normalized values of the row data x_n^t.

$$\tilde{x}_n^t = \frac{x_n^t - \mathbb{E}[x^t]}{\text{std}[x^t]} \tag{5}$$

So, the normalized two-dimensional historical 1-time step input feature matrix can be represented in this way:

$$\tilde{\mathcal{X}}_n^t = \{\tilde{x}_n^{t-m} | m = 19, 18, \dots, 0\} \tag{6}$$

In this way, the input sequences are normalized with zero mean and unit variance, which aims to reduce the influence from outlying time points and allow different features to be comparable with each other (Klambauer et al., 2017).

Target labels

For a stock set S^t on the trading timestamp t, each stock s_n^t has a profit rate r_n^{t+1} on the trading timestamp $t+1$, where the calculation method is the same as for r_n^t in the sequence of inputs shown in Eq. (3). The set $\mathcal{G}^{t+1}$ contains the next-time stamp's profit for N stocks in the timestamps $t+1$. It is defined as $\mathcal{G}^{t+1} = \{r^{t+1}\}_{n=1}^N$. Each r_n^{t+1} in the list $\mathcal{G}^{t+1}$ is then ranked by $\Psi(r_n^{t+1})$, which is the empirical quantile CDF for r_n^t:

$$\Psi(r_n^{t+1}) = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}_{\{r_i^{t+1} \le r_n^{t+1}\}} \tag{7}$$

For the target label for each feature s_n^t in $\mathcal{G}^{t+1}$, we have a one-hot vector $y_n^t \in \mathbb{R}^\varrho$ based on $\Psi(r_n^{t+1})$. y_n^t is partitioned into ϱ equal bins, where ϱ is set to 3 or more. Each bin corresponds to a $\varphi \times 100\%$ of the stocks in the set, where $\varphi\varrho \le 1$. And the boundary term ξ is used to ensure non-overlapping intervals. Then y_n^t is represented as:

$$y_n^t = [\mathbb{1}_{\{(i-1)(\varrho+\xi) \le \Psi(r_n^{t+1}) < i\varrho + (i-1)\xi\}} | i = 1, \dots, \varrho] \in \mathbb{R}^\varrho, \quad \xi = \frac{1-\varphi\varrho}{\varrho - 1} \tag{8}$$

For the empirical quantile CDF $\Psi(r_n^{t+1})$ of the stock in the range $[i\varphi, (i+1)\xi)$ for $i = 1, \dots, \varrho$, y_n^t is represented as $\mathbf{0}_\varrho$. For example, suppose $\varrho = 3$, $\varphi = 0.2$; for the stocks s_n^t whose profit values r_n^t ranked in the top, middle, bottom $\varphi \times 100\% = 20\%$ respectively, as ($\xi = 0.2$ in this case):

$$y_n^t = \begin{cases} [1,0,0]^T & \text{if } \Psi(r_n^{t+1}) \in [0, 0.2) \\ [0,1,0]^T & \text{if } \Psi(r_n^{t+1}) \in [0.4, 0.6) \\ [0,0,1]^T & \text{if } \Psi(r_n^{t+1}) \in [0.8, 1.0) \\ [0,0,0]^T & \text{otherwise} \end{cases} \tag{9}$$

The stocks in the intermediate ranges $(\Psi(r_n^{t+1}) \in [0.2, 0.4), [0.6, 0.8))$ are marked as $[0,0,0]^T$. Alternatively, when $\varrho = 5$ and $\varphi = 0.2$, the stocks are divided into five equally sized quantiles, with no null labels ($[0,0,0,0,0]^T$), where each part contains 20% of the stock in S^t:

$$y_n^t \in \{[1,0,0,0,0]^T, [0,1,0,0,0]^T, \dots, [0,0,0,0,1]^T\} \tag{10}$$

3.3. Quantformer encoder

Then, a quantformer encoder structure is designed as described in this subsection. The representation hierarchy consists of L blocks of multi-head self-attention layers. Taking initialized stock embedding sequences $\mathcal{X} = \{\tilde{\mathcal{X}}_n^{(t)}\}_{n=1}^N \in \mathbb{R}^{N\times 20\times 2}$ as inputs, the canonical self-attention layers introduced in Vaswani et al. (2017) can perform information exchange between every time points for each stock s_n^t.

To process both categorical and numerical data, the word embedding layer is replaced by a standard linear layer, utilizing linear transformations to substitute the process of word embedding. The linear embedding is shown in Eq. (11), where $\mathbf{W}_E \in \mathbb{R}^{2\times d}$ is the trainable weight matrix and $\theta_E \in \mathbb{R}^d$ is the bias and d is the dimension of the hidden feature space. The resulting sequence $\mathcal{X}_i'$ is

$$\mathcal{X}_i' = \mathcal{X}_i \mathbf{W}_E + \theta_E \tag{11}$$

In stock prediction, we aim to accurately forecast the return for a future period, thus the model's output is generally a single value representing the probability of price increase or decrease. Therefore, the decoder is simplified by removing the autoregressive prediction mechanism (in the decoder) and the masking operations. For the attention head $h = 1, \dots, H$, the query, key, and value metrics for each $\mathcal{X}_i \in \mathbb{R}^{20\times 2}$ are computed using separate learned linear projections:

$$\mathbf{Q}_{i,h} = \mathcal{X}_i' \mathbf{W}_h^Q, \quad \mathbf{K}_{i,h} = \mathcal{X}_i' \mathbf{W}_h^K, \quad \mathbf{V}_{i,h} = \mathcal{X}_i' \mathbf{W}_h^V \tag{12}$$

where the trainable weights $\mathbf{W}_h^Q$, $\mathbf{W}_h^K$, $\mathbf{W}_h^V \in \mathbb{R}^{d\times\varrho}$ for the $h-$th head. Then, the attention parameter for each head is computed using a scaled dot-product attention:

$$\text{Attention}(\mathbf{Q}_{i,h}, \mathbf{K}_{i,h}, \mathbf{V}_{i,h}) = \text{softmax}(\mathbf{Q}_{i,h}\mathbf{K}_{i,h}^T/\sqrt{d})\mathbf{V}_{i,h} \tag{13}$$

The outputs from all heads are concatenated and sent back to the original dimension using $\mathbf{W}^O \in \mathbb{R}^{\rho H \times d}$, and the final multi-head attention output $\mathbf{F}_i$:

$$\mathbf{F}_i = \text{Multihead}(\mathbf{Q}_{i,h}, \mathbf{K}_{i,h}, \mathbf{V}_{i,h}) = \left(||_{h=1}^{h=H} \text{Attention}(\mathbf{Q}_{i,h}, \mathbf{K}_{i,h}, \mathbf{V}_{i,h})\right)\mathbf{W}^O \tag{14}$$

where || represents the concatenation operator. After applying attention to all inputs, all the output from attention modules are represented in the form of:

$$\mathbf{F} = [\mathbf{F}_1; \mathbf{F}_2; \dots \mathbf{F}_N] \in \mathbb{R}^{N \times 20 \times \rho H} \tag{15}$$

which are fed into feed-forward layers.

3.4. Output process

After processing through the multi-head self-attention and feed-forward layers, with the encoder output F_i, the output layer can be represented as:

$$\mathbf{Z}^t = \text{Softmax}(\mathbf{F}_i \mathbf{W}_Z + \theta_Z) = \{z_1^t, z_2^t, \dots, z_N^t\} \in \mathbb{R}^{\rho} \tag{16}$$

where $\mathbf{W}_Z \in \mathbb{R}^{\rho \times d}$ and $\theta_Z \in \mathbb{R}^d$. With the softmax function, the predicted probability of the output can be represented as:

$$\hat{Y}_n^t = \{\hat{y}_{n,i}^t\}_{i=1}^{\rho} = \left\{ \frac{\exp(z_i^t)}{\sum_{i=1}^{\rho} \exp(z_i^t)} \right\}_{i=1}^{\rho} \in \mathbb{R}^{\rho} \tag{17}$$

In Eq. (17), the $\hat{Y}_n^t$ take values between 0 and 1 and sum to 1, which means that they can be interpreted as the probability distribution of the stock's performance.

3.5. Prediction

The Mean Squared Error Loss (MSELoss) is used to quantify the loss. It is defined as the average of the squares of the differences between the predicted value and the actual prices:

$$\text{MSELoss} = \frac{1}{N} \sum_{i=1}^{N} ||(y_i^t - \hat{y}_i^t)||_2^2 \tag{18}$$

where $||\cdot||_2^2$ denotes the squared Euclidean distance between predicted and target class probability vectors.

4. Experiments

In this section, we test the ability of the quantformer architecture, introduced in the previous section, for stock price prediction. We are going to detail the set of experiments, including the data resource, implementation details, trading strategy and metrics.

4.1. Dataset

Our training data contains 4601 stocks listed either on the Shanghai Stock Exchange (SHSE) or on the Shenzhen Stock Exchange (SZSE). The time period ranges from January 2010 to May 2023. The data has been obtained from AKShare[1] and Tushare.[2] which are quantitative finance terminals. The training period is from January 2010 to December 2019 and the testing period starts from January 2020. Recalling Section 3.2, for each stock, the input features consist of a rolling window of length 20, where each time step includes two variables: the accumulated return and the accumulated turnover rate.

Closing price adjustments, such as dividends, stock splits, and other corporate actions that can affect a stock's price, are applied to the stock training prices. These adjustments are essential for stock price analysis, especially on long time period, as they provide a more accurate picture of a stock's value and performance over time, which is more appropriate for financial backtesting. (Diamond, 1971; Wei et al., 2022).

Table 2
Detailed information about experiments.

Strategy	Frequency	Label dim(ρ)	Training samples	Section	Null-label
Month_1	Monthly	3	85,490	100	w/o
Month_2	Monthly	3	142,409	100	w/
Month_3	Monthly	5	142,409	100	w/o
Week_1	Weekly	3	455,157	466	w/o
Week_2	Weekly	3	758,300	466	w/
Week_3	Weekly	5	758,300	466	w/o
Day_1	Daily	3	3,586,435	2420	w/o
Day_2	Daily	3	5,140,279	2420	w/
Day_3	Daily	5	5,140,279	2420	w/o

4.2. Time frequency

Data frequency refers to the number of data points within a specific unit of time, which may reflect different market characteristics (De Prado, 2018). We consider three frequencies: monthly, weekly, and daily stock data.

In the first experiments, the time frequency is set to one month. For a stock s_n^t from the stock set S^t, each item in the feature sequence contains the accumulated profit and accumulated turnover rate in the month (Eq. (4)). Sometimes, the available data may not cover whole trading days, for example, because a stock was first listed on the market or resumed trading after an interruption. In such a situation, we would still record the stock data for this day. However, if there is a whole month during which the stock did not have any trading (perhaps due to stopped trading or not being listed in that month), this month's data will be recorded as "NaN", and the sequence $\mathcal{X}_n^t$ will not be used to train the model.

Within the first experiment, three sub-experiments are set. The sub-experiments share the same inputs. The values $\rho = 3$ and $\rho = 5$ are used in this experiment. The value of φ is fixed for all the experiments to 0.2. In the groups of $\rho = 5$, stocks are divided into five parts and marked respectively. Then, in the groups of $\rho = 3$, the top 20%, middle 20% and bottom 20% of the stocks are labeled with one-hot vectors $[1, 0, 0]$, $[0, 1, 0]$ and $[0, 0, 1]$, respectively. Stocks in the intermediate quantile (i.e., $20\% - 40\%$ and $60\% - 80\%$) are marked as $[0, 0, 0]$ (as the boundary term $\xi = 0.2$ here, and $\varphi = 0.2$ as well). In the first group of $\rho = 3$, the null labels ($[0, 0, 0]^T$) are excluded from training, whereas the second group of $\rho = 3$ includes them.

Outliers of both accumulated profit and accumulated turnover rate are not removed from the dataset. These situations may happen in the future and they are expected to be predicted, though these incidents rarely happen.

In the second and third experiments, the inputs are in weekly and daily frequency, respectively. The sub-experiments under different groups are similar to the first one. All the accumulated number of parameters, the number of trained sections (such as 100 months from 2010 to 2019), and output dimensions are shown in Table 2.

4.3. Implementation details

The model and its training process are implemented using PyTorch (Paszke et al., 2019). The hyperparameters of the model are optimized via grid search as is simple to implement and reliable in low-dimensional

[1] https://akshare.akfamily.xyz/
[2] https://www.tushare.pro/

search spaces (Bergstra & Bengio, 2012). In our setting, the hyperparameter space is intentionally constrained due to the simplified transformer model, and the grid search is conducted offline as a one-time procedure with a limited number of configurations. Here, the input dimension is 2, and the dimension of the hidden feature space d is set to 16. The model uses 16 attention heads, with 6 layers in both the encoder and decoder. Training is performed using the Adam optimizer (Kingma & Ba, 2015) for 50 epochs, with a learning rate of 0.001 and a batch size of 64. All experiments are conducted on a workstation equipped with an Intel Core i5-10300H CPU, 16 GB of RAM, and an NVIDIA GeForce RTX 2070 GPU with 8 GB of memory. Under this setup, a single model training typically takes approximately 2–5 hours (depending on the frequency), while the full backtesting procedure can be completed within about 1 h.

4.4. Trading strategy

Algorithm 1 shows the pseudocode of the trading strategy. Before the first trade date of the timestamp t, all sequences $\mathcal{X}_n^t$ from the stock set S^t are put in the model and the list of outputs $\hat{Y}^t$ is required. The portfolio value at time t, denoted as P^t is updated based on the previous period's weights and returns as follows:

$$P^t = P^{t-1}\left(\sum_{n=1}^{N} w_n^{t-1}(1 + r_n^{t-1})\right) \tag{19}$$

where w_n^t is the weight of stock s_n^t, r_n^t is the return of the stock, and $\sum_{n=1}^{N} w_n^t = 1$. To determine the weight of each stock, a trading strategy $\Phi \in \mathbb{R}^{\varrho}$ and the decision factor $\mathbf{b}$ is used, where $\mathbf{b}$ determines how many quantile groups are selected, satisfying $1 \le \mathbf{b} < \varrho$. The strategy does not allow short selling (as the Chinese stock market does not allow short selling), so Φ only contains 0 or 1.

$$\Phi \in \{0,1\}^{\varrho}, \quad ||\Phi||_0 = \mathbf{b} \tag{20}$$

For example:

- If $\varrho = 3$, $\mathbf{b} = 1$, then $\mathbf{\Phi}$ could be one of: $[1,0,0]$, $[0,1,0]$, or $[0,0,1]$.
- If $\mathbf{b} = 2$, then possible values of $\mathbf{\Phi}$ include: $[1,1,0]$, $[1,0,1]$, or $[0,1,1]$.

Recall the predicted output $\hat{y}_n^t = \{\hat{y}_{n,i}^t\}_{i=1}^{\varrho}$, we sort S^t by $\Psi(\hat{y}_{n,1}^t)$, which is the empirical quantile CDF for $\hat{y}_{n,1}^t$. Similar to the Eq. (7), $\Psi(\hat{y}_{n,1}^t)$ is computed in this way:

$$\Psi(\hat{y}_{n,1}^t) = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}_{\{\hat{y}_{i,1}^t \le \hat{y}_{n,1}^t\}} \tag{21}$$

The sorted predicted vector $\tilde{y}_n^t$ for stock s_n^t at time t is shown below. Here the stocks are ranked into ϱ parts and each part contains $\varphi \times 100\%$ of the stocks, and the parameters ϱ and φ satisfy $\varphi\varrho \le 1$:

$$\tilde{y}_n^t = [\mathbb{1}_{\{(i-1)(\varrho+\xi) \le \Psi(\hat{y}_{n,1}^t) < i\varrho + (i-1)\xi\}} | i = 1, \ldots, \varrho] \in \mathbb{R}^{\varrho}, \quad \xi = \frac{1-\varphi\varrho}{\varrho - 1} \tag{22}$$

Then the weight for each selected stock is computed using Eq. (23) shown below, and all the chosen stocks are equal-weighted in this way:

$$w_n^t = \frac{1}{\mathbf{b}\cdot\varrho}(\tilde{y}_n^{t-1}\Phi^T\tilde{y}_n^t)^T \cdot \mathbf{1} \tag{23}$$

The same method is run repeatedly during the subsequent periods. The backtest starts from January 2020, in other words, the result of the sequences from May 2018 to December 2019 will be used as the first stock pool to trade. The exchange fee in the Chinese stock market is 0.00384% (Shanghai stock exchange, 2023; Shenzhen stock exchange, 2023), and the brokerage commission is capped at 0.3% of the transaction amount (China securities regulatory commission, 2002). To keep our backtest results conservative, we set the transaction fee in the backtest to 0.3%.

Algorithm 1 Trading strategy.

Require: The feature sequence χ_n^t for each stock $s_n^t \in S^t$, initial portfolio cash P^0, trading strategy Φ, strategy decision factor $\mathbf{b}$.

```
run_frequency(trade, frequency, startday = 1, time = 'open')
function PREDICT(χ_n^t): return ŷ_n^t = quantformer(χ_n^t)
end function
function SORTLABEL(ŷ_{n,1}^t): return ỹ_n^t = [1{ (i-1)/ϱ ≤ Ψ(ŷ_{n,1}^t) < i/ϱ }]_{i=1}^{ϱ}
end function
function COMPUTEWEIGHT(ỹ_n^{t-1}, ỹ_n^t, Φ):
    numerator = ỹ_n^{t-1} · Φ^⊤ · ỹ_n^t
    denominator = Σ_{k=1}^{N} ỹ_k^{t-1} · Φ^⊤ · ỹ_k^t
    return w_n^t = 1/(b·ϱ) · numerator/denominator
end function
function TRADE(w_n^t, P^t):
    for s_n^t in S^t do order(s_n^t, w_n^t · P^t)
    end for
end function
```

4.5. Metrics

The Sharpe Ratio (SR) (Sharpe, 1964) and the α rate will be used to test the performance of the strategy. The SR is a measure of risk-adjusted return that describes the additional earnings an investor receives for each standard deviation unit increase (Eq. (24)), where R_p is the return of the portfolio and R_f is the risk-free rate, chosen to be the London Interbank Offered Rate (LIBOR) which averages rate estimates submitted by banks in London[3]. The risk-free rate in the backtest will be calculated as the average LIBOR rate during the test period.

$$\text{SR} = \frac{\mathbb{E}[R_p] - R_f}{\text{std}[R_p]} \tag{24}$$

Alpha represents the excess return of an investment portfolio relative to its benchmark. It aims to measure the stock selection skills of an investment strategy. A positive alpha indicates that the investment portfolio has achieved a higher return than its benchmark after risk adjustment. Specifically, alpha is the excess return of the actual portfolio return over its expected theoretical return. Eq. (25) shows the calculation method of the alpha rate, where $E(R_m)$ is the market return and β is the correlation between the portfolio and the benchmark's systemic risk.

$$\alpha = (R_p - R_f) - \beta(\mathbb{E}[R_m] - R_f) \tag{25}$$

Besides the SR ratio and alpha rate, the annual return (AR) of the stock, the annual excess return (AER), the win rate (WR) and the average turnover rate (TR) of the portfolio will also be shown. The excess returns are returns achieved above and beyond the return of a proxy, chosen to be the CSI 300 index. The win rate is the percentage of trading days during which the portfolio strategy generates a positive return. The turnover rate of the portfolio is the proportion of portfolio weight changes between two consecutive rebalancing periods. The definition of TR at each time t is:

$$\text{TR}^t = \frac{1}{2}\sum_{k=1}^{N} |w_n^t - w_n^{t-1}| \tag{26}$$

where w_n^t and w_n^{t-1} denote the portfolio weight of the stock s_n in the portfolio at time t and $t-1$, respectively.

Value at risk (VaR) is a method to summarize the total risk in a portfolio (Hull, 2012).

$$\text{VaR}_{\alpha} = \inf\{x : P(\mathcal{L} > x) \le \alpha\} \tag{27}$$

[3] Although LIBOR officially came to an end on September 30th, 2024, the time period covered by our backtesting dataset is prior to this date. https://www.bankofengland.co.uk/news/2024/october/the-end-of-libor

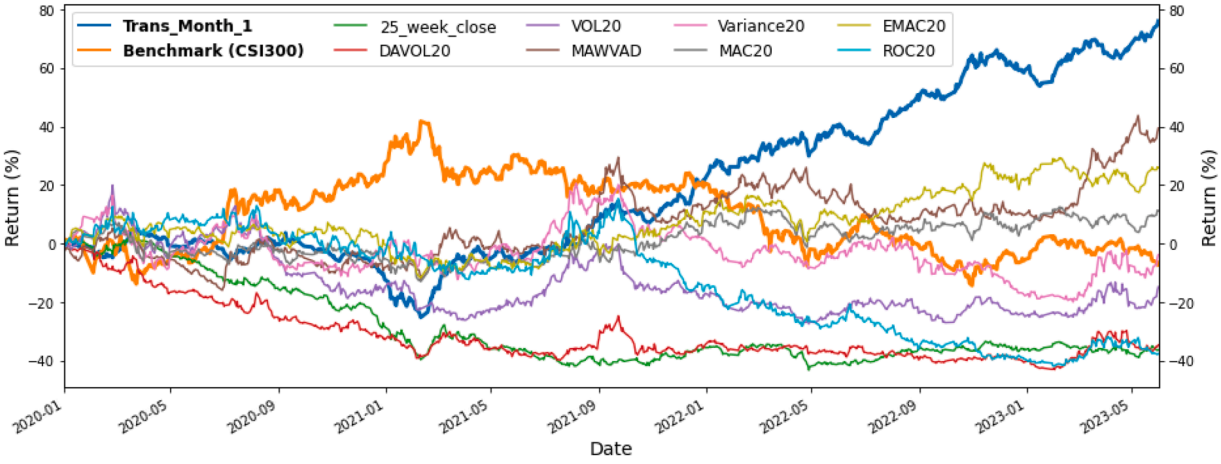


Fig. 4. Backtest results based on different factors.

Eq. (27) shows the calculation of VaR, where $\mathcal{L}$ is the loss of the holding period T, then VaR(α) is the αth upper quantile of $\mathcal{L}$. In the measurement of the portfolio, 99% VaR is used to estimate the maximum loss during the period with 99% confidence.

5. Results and discussion

Building upon the analysis of individual strategies, this section presents an evaluation and discussion of the experimental outcomes. The discussion focuses on the comparative effectiveness of different time frequencies (monthly, weekly, and daily) and different training scales.

5.1. Overall performance

Our results are shown in Table 3. For the monthly group, Month_1 shows an annual return of 17.35% and an annual excess return of 19.43%, indicating a robust performance over that period. Conversely, Month_3 shows a diminished annual return and annual excess return of 7.37% and 9.91% respectively. The is notably high in the second month at 51.69%. The win rate ranges from 43.1% to 57.3%, with the first month showing the highest win rate. The Sharpe Ratio and Alpha vary across the strategies, with some periods showing negative values, suggesting underperformance relative to the risk taken, while others show positive values. In the comparison of 99% VaR, the strategies in monthly frequency perform better than others, where Month_1 and Month_3 are lower than 3%. This means that, with 99% confidence, the portfolio maximum daily losses are 2.81% and 2.3% respectively.

In the weekly strategy results, Week_3 shows an annual return of 12.3% and an annual excess return of 12.73%, while Week_1 displays a negative annual return. For the daily strategies, Day_2 has a return of 10.23% and an annual excess return of 10.94%. The turnover rate across the daily strategies shows lesser variation than the monthly or weekly, indicating a more consistent trading frequency. The win rate for the daily strategies remain fairly stable, hovering around the mid-40% mark. The Sharpe Ratio and Alpha for weekly and daily strategies show a mix of positive and negative values, reflecting the fluctuating nature of shorter-term trading efficacy.

To demonstrate the advantages of the quantformer factor over traditional factors, 100 price-volume type factors from JoinQuant [4] (one of China's largest quantitative finance platforms) are selected for backtesting under the same trading strategy (Algorithm 1). Price-volume factors are used because these are calculated based on stocks' prices, volumes, and turnover rates, which are similar to the training data used by quantformer factors. The detailed performance of factors and benchmark is shown in Appendix A. On average, these factors achieves an annual return of -3.78%, an average excess return of -2.15%, and a Sharpe ratio of -0.36, where the benchmark return is -1.77%. The average maximum drawdown of the strategy is 44.88%, and the average volatility of the strategy (σ_p) is 0.26, with the benchmark at 0.197.

Among these factor-based strategies, the quantformer factor performs well. Except for the win rate, which is slightly lower than a few strategies, it performs well in terms of annual return, annual excess return, σ_p, Sharpe ratio and Sortino ratio, ranking the best among the 101 factors. The 99% VaR of the quantformer factor is among the best 10% factors. Fig. 4 illustrates the return curves of the quantformer (QF_Month_1) alongside the benchmark and some traditional factors. "25_week_rank" represents the current price's position over the past 25 weeks; "EMAC20" and "MAC20" are the 20-day index moving aver-

Table 3
Result of experiments.

Strategy	AR	AER	TR	WR	SR	Alpha	VaR
Month_1	**17.35%**	**19.43%**	26.09%	**57.8%**	**0.915**	**0.162**	2.81
Month_2	9.91%	13.86%	**51.69%**	49.3%	0.289	0.102	3.61
Month_3	7.37%	9.91%	32.33%	51.6%	0.246	0.064	**2.3**
Week_1	-0.83%	1.31%	**7.13%**	46.4%	-0.236	-0.030	3.05
Week_2	7.49%	10.81%	1.18%	49.2%	0.160	0.085	3.73
Week_3	**12.3%**	**12.73%**	1.39%	**54.4%**	**0.372**	**0.116**	3.77
Day_1	7.89%	**11.4%**	**6.71%**	43.1%	0.181	0.090	3.92
Day_2	**10.23%**	10.94%	6.51%	44.4%	**0.279**	**0.097**	3.91
Day_3	9.81%	10.03%	**5.57%**	**44.6%**	0.281	0.092	4.02
CSI300	1.77%	\	\	\	-0.015	\	3.19%

Table 4
Result of factors under different scales.

Strategy	AR	AER	σ_p	WR	SR	Alpha	VaR
QF_20%	17.35%	19.43%	0.162	57.8%	0.915	0.162	2.81%
QF_10%	13.12%	17.73	0.159	**61.8%**	0.574	0.128	2.023%
QF_5%	12.59%	16.02%	**0.136**	61%	0.63	0.117	**2.015%**
QF_1%	**24.71%**	**35.74%**	0.214	53.3%	**0.967**	**0.249**	3.048%
CSI300	1.77%	\	0.197	\	-0.015	\	3.19%

[4] https://www.joinquant.com/

age and the stock's 20-day moving average respectively; "DAVOL20" and "VOL20" represent the ratio of the 20-day average turnover rate to the 120-day average turnover rate, and the mean of the stock's 20-day turnover rate; "ROC20" is the price rate of change over 20 days; "MAWVAD" is calculated as the product of the difference between the closing price and the opening price, divided by the range of the highest and lowest prices, all multiplied by the volume, accumulated over six days; "Variance20" is the variance of the stock's 20-day annualized returns. Most of these factors are compared based on a window of 20 timestamps, as the quantformer factor was also trained on 20 timestamps. In comparison with these factors, the quantformer factor (blue line) demonstrates a significantly better performance in terms of returns than the benchmark (orange line) and other factor strategies. Compared to other factors by the Sharpe ratio, the quantformer (0.915) still performs better than others (the highest Sharpe ratio among these factors is 0.243). Furthermore, the quantformer also shows a lower downside risk in terms of risk assessment as it achieves a VaR of 2.81, compared to the highest VaR of 2.86 among other factors. These indicate the improvement of the quantformer factor over traditional price-volume factors.

5.2. Training under different scales

To test the stock selection capabilities of the model, three other factors "QF_10%", "QF_5%", "QF_1%" are trained and tested (with "QF_Month_1%", named as "QF_20%"). The four groups are all trained under $\rho = 3$, which include the first, middle, and tail 20%, 10%, 5%, and 1% of stocks in ranking, respectively. The backtest period and other settings are the same as in Section 4.

Table 4 shows the result of the backtest. QF_10% strategy records an AR of 13.12% and an AER of 17.73%, with a slightly lower portfolio volatility of 0.159. This strategy has the highest WR at 61.8%. QF_5% strategy delivered 12.59% AR and 16.02% AW, with the lowest portfolio volatility at 0.136. It has the lowest VaR at 2.015%. QF_1% strategy outperformed the others, achieving the highest AR of 24.71% and the highest AER of 35.74%, though it takes the highest volatility at 0.214. These four factors all perform better than CSI 300 based on their return and risk. These results suggest that the quantformer model's performance seems robust to the choice of different training sets and requirements.

6. Conclusion

This study proposes a new neural network architecture, *quantformer*, inspired by the transformer architecture, for quantitative stock prediction and trading. We address the need for handling numerical financial input data rather than text, and adapted the model for forecasting tasks rather than sequence-to-sequence problems common in NLP. To enable direct processing of numerical time series data, we replace the word embedding layer with a standard linear layer and remove the output masking operations. We also simplify the decoder to produce a probability distribution over future price movements rather than autoregressively generating token sequences.

We also backtest the quantformer factor on financial market data and compare it with 100 other price-volume factors. Our experimental results demonstrate the promise of this approach. The quantformer-based trading strategies are able to deliver substantial excess returns over the benchmark under a longer time period, with Sharpe Ratios indicating sound risk-adjusted performance. The positive alphas affirm the strategy's ability to outperform expected returns after accounting for risk factors.

For future work, extending quantformer to more quantitative finance scenarios, such as execution problems or limit order book or accepting more flexible input types beyond price-volume features may be valuable directions. The current study primarily focuses on daily, weekly and monthly frequency of financial time series and is trained on large-scale backtesting over a long time period, which is computationally expensive.

Overall, our work illustrates the viability of the quantformer architecture to handle financial time series, design profitable and robust trading strategies, and calls for further analysis and investigations of this proposed ML-based quantitative methods. The implementation code of quantformer is available at https://github.com/zhangmordred/QuantFormer.

CRediT authorship contribution statement

Zhaofeng Zhang: Conceptualization, Methodology, Software, Resources, Software, Writing – original draft, Writing – review & editing, Visualization; **Banghao Chen:** Software, Validation, Resources, Writing – original draft; **Shengxin Zhu:** Resources, Writing – review & editing, Supervision, Funding acquisition; **Nicolas Langrené:** Methodology, Writing – review & editing, Supervision, Funding acquisition.

Data availability

The authors do not have permission to share data.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was funded by the National Key Research and Development Program of China (2025YFG0202100); National Natural Science Foundation of China (12271047); Guangdong Provincial/Zhuhai Key Laboratory of Interdisciplinary Research and Application for Data Science, Beijing Normal-Hong Kong Baptist University (2022B1212010006); Beijing Normal-Hong Kong Baptist University research grants (UICR0400008-21; UICR0700041-22; R72021114); and the Guangdong University Innovation and Enhancement Program (2021ZDZX1046).

Appendix A. Detailed backtest results between 100 factors and the quantformer factor

All the data of factors come from JoinQuant,[5] which is one of the most common quantitative finance platforms in China. Detailed descriptions of the following factors are available at https://www.joinquant.com/help/api/help#factor_values.
Ten metrics are used to evaluate the performance of each factor:

- Annual Return (**AR**) and Annual Excess Return (**AER**) are two metrics to reflect the return of the portfolio directly. The excess returns are returns achieved above and beyond the return of a proxy and the CSI 300 index is used as the basis when calculating excess returns.
- Win Rate (**WR**) is the metric defined as the fraction between the number of trading periods that generate a profit over the total number of trading periods.
- Sharpe Ratio (**SR**) is a measure of risk-adjusted return that describes the additional earnings an investor receives for each unit of the standard deviation of returns. It is shown in Eq. (24).
- The **Alpha** measures the ability of a portfolio to generate returns above the market benchmark. A positive alpha indicates that the investment portfolio has achieved a higher return than its benchmark after risk adjustment. It is shown in Eq. (25).
- **Beta** is the correlation between the portfolio and systemic risk, reflecting the sensitivity of the strategy to changes in the market. For the daily return of the strategy, D_p, and the daily return of the benchmark D_m, the equation of beta is:

$$\text{Beta} = \beta_p = \frac{\mathbb{C}\text{ov}(D_p, D_m)}{\mathbb{V}\text{ar}(D_m)} \tag{A.1}$$

  where $\mathbb{C}\text{ov}(D_p, D_m)$ is the correlation between D_p and D_m, and $\mathbb{V}\text{ar}(D_m)$ is the variance of the daily returns of the benchmark.
- Max Drawdown (**MD**) means the potential worst-case scenario or the most extreme possible loss from the previous peak. For the trough value of a portfolio A_{trough} and the previous peak value of the portfolio A_{peak}, the MD is calculated in the following way:

$$\text{MD} = \frac{A_{\text{peak}} - A_{\text{trough}}}{A_{\text{peak}}} \tag{A.2}$$

- Portfolio volatility (σ_p) is the standard deviation of the portfolio returns. It provides a kind of weighted average deviation in which large deviations carry more weight (Bacon, 2023).
- Sortino Ratio (**STN**) differentiates harmful volatility from total overall volatility by using the standard deviation of negative portfolio returns. This metric measures the performance of the investment relative to the downward risk.
- Value-at-Risk (**VaR**) estimates the potential loss in value of a risky investment, which is used to quantify the amount of potential loss and the likelihood of occurrence for that loss within a specified time frame, which is shown in Eq. (27). In the backtest, 99%VaR is used.

Factor	AR (%)	AER (%)	WR (%)	SR	Alpha	Beta	MD (%)	σ_p	VaR (%)	STN
ARBR	−1.47	0.32	54.7	−0.251	−0.002	0.914	34.46	0.221	3.74	−0.375
AR	−5.87	−4.37	53.7	−0.406	−0.048	0.981	47.84	0.260	3.94	0.612
ATR14	−2.12	−0.37	62.7	−0.250	0.006	1.160	46.55	0.250	4.51	−0.35
ATR6	−3.94	−2.31	61.2	−0.328	−0.014	1.175	50.11	0.253	4.48	−0.455
BBIC	1.17	3.14	50.9	−0.109	0.033	1.038	32.64	0.258	4.16	−0.155
BR	−6.91	−5.47	53.6	−0.467	−0.064	0.934	51.72	0.254	3.80	−0.718
CCI10	−18.65	−17.99	43.3	−1.402	−0.261	0.812	72.04	0.220	2.79	−2.026
CCI15	−17.70	−16.98	44.2	−1.293	−0.240	0.832	71.43	0.223	2.90	−1.866
CCI20	−15.52	−14.66	46.3	−1.051	−0.196	0.868	69.12	0.235	3.28	−1.519
CR20	−4.27	−2.67	54.8	−0.329	−0.032	0.927	49.92	0.263	4.26	−0.511
DAVOL10	−7.44	−6.04	54.1	−0.504	−0.071	0.920	45.45	0.248	3.38	−0.812
DAVOL20	−10.95	−9.78	54.4	−0.702	−0.119	0.919	48.96	0.246	2.86	−1.079
DAVOL5	−9.54	−8.28	51.8	−0.612	−0.097	0.950	46.41	0.250	3.17	−1.019
EMA5	2.77	4.85	48.3	−0.047	0.049	1.041	30.70	0.259	4.66	−0.068
EMAC10	0.27	2.17	50.6	−0.144	0.024	1.037	31.42	0.258	4.19	−0.203
EMAC12	1.58	3.57	50.6	−0.093	0.037	1.032	31.76	0.258	4.17	−0.131
EMAC20	6.01	8.29	54.1	0.069	0.079	1.044	32.43	0.260	4.32	0.102
EMAC26	3.83	5.97	54.1	−0.008	0.059	1.046	34.87	0.261	4.29	−0.012
EMAC120	1.74	3.75	48.3	−0.088	0.034	0.960	35.81	0.254	3.77	−0.145
Kurtosis20	−2.96	−1.27	52.5	−0.395	−0.027	0.772	27.71	0.182	2.74	−0.6
Kurtosis60	−0.28	1.59	53.9	−0.240	−0.001	0.716	22.07	0.179	2.64	−0.367
Kurtosis120	−4.12	−2.51	49.0	−0.468	−0.043	0.708	28.28	0.182	2.44	−0.684
MAC5	0.54	2.46	46.9	−0.134	0.026	1.035	30.03	0.257	4.29	−0.195
MAC10	−1.20	0.61	49.6	−0.206	0.008	1.039	34.15	0.256	4.19	−0.286
MAC20	2.47	4.52	53.5	−0.059	0.046	1.043	34.58	0.257	4.09	−0.083
MAC60	11.10	13.72	53.8	0.243	0.120	0.987	28.61	0.254	4.34	0.399
MAC120	3.11	5.21	47.7	−0.035	0.049	0.987	35.40	0.259	3.99	−0.058

[5] https://www.joinquant.com/

MACDC	−11.12	−9.97	52.4	−0.674	−0.117	1.003	46.48	0.261	3.54	−1.046
MASS	−10.55	−9.35	51.0	−0.673	−0.115	0.886	48.34	0.249	6.19	−1.032
MAWVAD	7.29	9.67	55.9	0.119	0.082	0.891	36.54	0.248	4.19	0.184
MFI14	−6.21	−4.73	51.5	−0.456	−0.056	0.918	45.65	0.241	3.61	−0.699
PLRC6	−14.83	−13.92	47.8	−0.905	−0.178	0.960	66.24	0.259	3.85	−1.368
PLRC12	−7.28	−5.87	53.3	−0.452	−0.064	1.012	52.30	0.273	4.20	−0.682
PLRC24	−5.77	−4.26	58.1	−0.361	−0.044	1.034	53.28	0.289	4.88	−0.549
PSY	−9.11	−7.83	50.6	−0.660	−0.096	0.880	49.79	0.223	3.36	−0.992
Price1M	−13.55	−12.55	53.0	−0.760	−0.156	0.977	64.85	0.280	4.21	−1.145
Price3M	−6.69	−5.25	58.4	−0.380	−0.056	1.023	59.14	0.305	5.44	−0.555
Price1Y	1.55	3.54	66.8	−0.075	0.041	1.116	60.25	0.326	7.28	−0.105
ROC6	−16.13	−15.30	47.4	−0.982	−0.200	0.978	68.14	0.262	3.56	−1.741
ROC20	−11.63	−10.51	54.7	−0.628	−0.123	1.014	60.28	0.291	4.51	−0.971
ROC60	−6.86	−5.43	59.6	−0.382	−0.056	1.062	62.16	0.309	5.28	−0.529
ROC120	2.36	4.41	66.9	−0.051	0.049	1.117	56.36	0.319	6.79	−0.071
Skewness20	−5.91	−4.41	50.8	−0.538	−0.058	0.820	40.65	0.197	3.04	0.839
Skewness60	−6.47	−5.01	51.9	−0.595	−0.068	0.765	38.02	0.190	2.58	−0.917
Skewness120	−4.37	−2.77	51.1	−0.459	0.044	0.743	36.01	0.192	2.69	−0.684
TRIX5	−10.72	−9.54	52.5	−0.607	−0.110	1.009	56.07	0.280	4.16	−0.923
TRIX10	−3.51	−1.85	58.4	−0.268	−0.019	1.008	52.92	0.292	5.14	−0.4
TVMA20	−1.92	−0.16	64.9	−0.209	0.014	1.268	44.66	0.288	4.63	−0.31
TVMA6	−0.75	1.09	62.4	−0.166	0.026	1.259	39.46	0.288	4.76	−0.246
TVSTD20	−1.83	−0.06	58.5	−0.214	0.012	1.218	35.89	0.277	4.03	−0.318
TVSTD6	−3.77	−2.13	53.4	−0.297	−0.010	1.202	46.57	0.272	4.40	−0.449
VDEA	−3.05	−1.37	57.6	−0.305	−0.023	0.857	37.01	0.239	3.06	−0.484
VDIFF	−7.93	−6.57	0.2	−0.533	−0.079	0.895	39.79	0.247	3.08	−0.842
VEMA5	−0.41	1.46	57.9	−0.186	0.006	0.850	37.31	0.237	3.33	−0.277
VEMA10	0.95	2.91	58.9	−0.128	0.019	0.844	35.51	0.236	3.36	−0.19
VEMA12	1.01	2.96	59.3	−0.126	0.020	0.840	35.17	0.235	3.35	−0.186
VEMA26	2.41	4.46	59.3	−0.068	0.033	0.838	31.03	0.231	3.37	−0.1
VMACD	−1.03	0.79	50.5	−0.216	0.001	0.875	33.96	0.235	3.38	−0.345
VOL5	−8.12	−6.76	55.9	−0.452	−0.066	1.155	53.86	0.296	4.48	−0.656
VOL10	−7.07	−5.64	58.3	−0.404	−0.053	1.145	50.65	0.298	4.35	−0.59
VOL20	−5.43	−3.89	58.6	−0.337	−0.032	1.156	50.19	0.298	4.33	−0.481
VOL60	0.42	2.34	58.1	−0.125	0.032	1.145	38.35	0.286	4.71	−0.175
VOL120	−1.88	−0.12	53.0	−0.213	0.007	1.136	42.09	0.281	4.64	−0.3
VOL240	−5.51	−3.98	47.0	−0.382	−0.038	1.075	43.53	0.265	3.99	−0.541
VOSC	−6.35	−4.88	51.2	−0.453	−0.056	0.944	40.40	0.246	3.44	−0.739
VROC6	−16.30	−15.49	42.7	−1.179	−0.208	0.897	69.03	0.221	2.91	−1.778
VROC12	−11.47	−10.34	46.7	−0.793	−0.127	0.915	51.62	0.228	2.74	−1.241
VR	−8.43	−7.10	52.8	−0.566	−0.083	0.932	48.57	0.244	3.72	−0.834
VSTD10	−0.77	1.07	56.3	−0.205	0.001	0.843	37.75	0.234	3.19	−0.309
VSTD20	−1.60	0.19	53.7	−0.245	−0.007	0.844	37.31	0.232	3.01	−0.366
Variance20	−1.94	−0.18	59.9	−0.201	0.005	1.124	46.38	0.302	4.84	−0.301
Variance60	9.96	12.51	65.2	0.168	0.122	1.178	30.63	0.313	5.91	0.241
Variance120	5.40	7.65	60.8	0.040	0.083	1.193	39.54	0.312	5.93	0.058
Volume1M	−15.54	−14.68	20.7	−0.867	−0.189	0.991	63.68	0.285	3.61	−1.339
WVAD	−0.71	1.13	52.3	−0.199	0.002	0.836	42.38	0.238	3.67	−0.317
arron_down_25	−7.35	−5.94	48.3	−0.593	−0.072	0.880	42.63	0.209	2.84	−0.826
arron_up_25	−15.51	−14.65	46.4	−1.049	−0.195	0.870	66.73	0.235	3.19	−1.604
bear_power	−13.31	−12.30	50.8	−0.800	−0.154	0.934	59.58	0.261	3.61	−1.182
beta	10.30	12.87	62.4	0.168	0.134	**1.344**	35.96	0.328	6.76	0.252
book_to_price	1.87	3.86	58.5	−0.117	0.031	0.889	26.73	0.181	2.91	−0.162
bull_power	−14.81	−13.91	51.1	−0.844	−0.176	0.984	65.99	0.278	3.83	−1.285
earnings_yield	−2.41	−0.68	52.6	−0.348	−0.029	0.627	25.87	0.188	**2.55**	−0.55
growth	2.90	4.99	59.9	−0.045	0.052	1.068	37.35	0.244	4.50	−0.067
leverage	2.05	4.07	53.1	−0.089	0.023	0.719	22.57	0.217	3.22	−0.144
liquidity	−2.71	−1.00	56.7	−0.243	−0.002	1.137	41.28	0.284	4.56	−0.35
momentum	10.93	13.54	65.8	0.195	0.129	1.159	43.19	0.310	6.82	0.271
money_flow_20	−1.72	0.05	65.5	−0.202	0.016	1.269	44.10	0.288	4.63	−0.298
price_no_fq	6.83	9.17	**76.1**	0.095	0.092	1.132	44.86	0.267	6.20	0.131
pull−up	2.19	4.23	54.5	−0.068	0.044	1.062	33.09	0.265	3.92	−0.093
pull−down	−2.28	−0.54	47.6	−0.317	−0.015	0.831	34.46	0.203	3.87	−0.439
residual_volatility	4.13	6.30	62.3	0.003	0.062	1.048	41.34	0.269	5.25	0.004

sharpe_ratio_20	−11.88	−10.78	54.0	−0.663	−0.129	0.979	57.66	0.282	4.09	−1.019
sharpe_ratio_60	−7.42	−6.02	58.8	−0.417	−0.065	1.029	65.19	0.300	5.17	−0.583
sharpe_ratio_120	−0.32	1.55	65.7	−0.143	0.019	1.062	60.52	0.303	6.10	−0.2
size	−6.36	−4.89	51.8	−0.571	−0.058	0.906	36.00	0.196	2.64	−0.82
turnover_volatility	−10.80	−9.62	51.8	−0.624	−0.108	1.075	57.05	0.274	3.70	−0.905
1day_VPT	−16.45	−15.65	44.8	−1.062	−0.211	0.904	70.43	0.248	3.09	−1.553
1day_VPT_6	−6.36	−4.89	53.0	−0.416	−0.053	0.995	47.06	0.269	3.81	−0.82
1day_VPT_12	−0.07	1.82	57.9	−0.146	0.020	1.033	46.51	0.278	4.39	−0.22
25week_close	−11.51	−10.39	51.0	−0.859	−0.133	0.817	47.79	0.211	2.44	−0.684
QF_Month_1	**17.35**	**19.43**	57.8	**0.915**	**0.162**	0.588	**18.35**	**0.161**	2.81	**0.946**
QF_Month_2	9.91	13.53	49.3	0.289	0.102	0.684	26.77	0.205	3.61	0.372
QF_Month_3	7.37	10.4	51.6	0.246	0.064	0.483	17.33	0.137	2.3	0.309
QF_Week_1	−0.83	1.41	46.4	−0.236	−0.003	0.726	29.31	0.204	3.05	−0.291
QF_Week_2	7.49	10.55	49.2	0.16	0.085	0.792	29.08	0.219	3.73	0.206
QF_Week_3	12.3	12.4	54.4	0.372	0.116	0.793	29.86	0.223	3.77	0.48
QF_Day_1	7.89	11.4	43.1	0.181	0.09	0.834	29.15	0.196	3.92	0.245
QF_Day_2	10.23	10.68	44.4	0.279	0.097	0.819	30.02	0.223	3.91	0.361
QF_Day_3	9.81	10.03	44.6	0.284	0.098	0.801	30.1	0.223	1.02	0.367
QF_10%	13.12	17.73	61.8	0.754	0.128	0.594	18.32	0.159	2.02	0.714
QF_5%	12.59	16.02	61	0.63	0.117	0.504	**13.3**	**0.136**	2.01	0.796
QF_1%	**24.71**	**35.34**	53.3	**0.967**	**0.249**	0.667	23.28	0.214	3.35	**1.299**
Average	−2.92	−1.25	53.28	−0.32	−0.02	0.96	43.61	0.25	3.99	−0.467
CSI300	1.77	\	\	0.009	\	\	\	0.197	3.19	\

Appendix B. Comparison between different quantformer strategies

In the provided scatterplot Fig. B.1, the QF_Month_1 factor exhibits notable performance in terms of annual return. Compared to other factors, QF_Month_1 offers better returns. Regarding risk, as measured by the Variance_Covariance_VaR_99, the QF_Month_1 factor's data points are positioned in a moderately high range, indicating a balanced risk profile. The size and color intensity of the QF_Month_1 points, reflecting higher Alpha values, suggest a robust excess return or relative performance. Combining these observations, the QF_Month_1 factor likely provides a compelling investment opportunity by delivering robust returns while maintaining a moderate risk level, an attractive proposition for strategies aiming to optimize the trade-off between risk and return. Overall, most of the factors based on quantformer perform better compared with the other 100 factors.

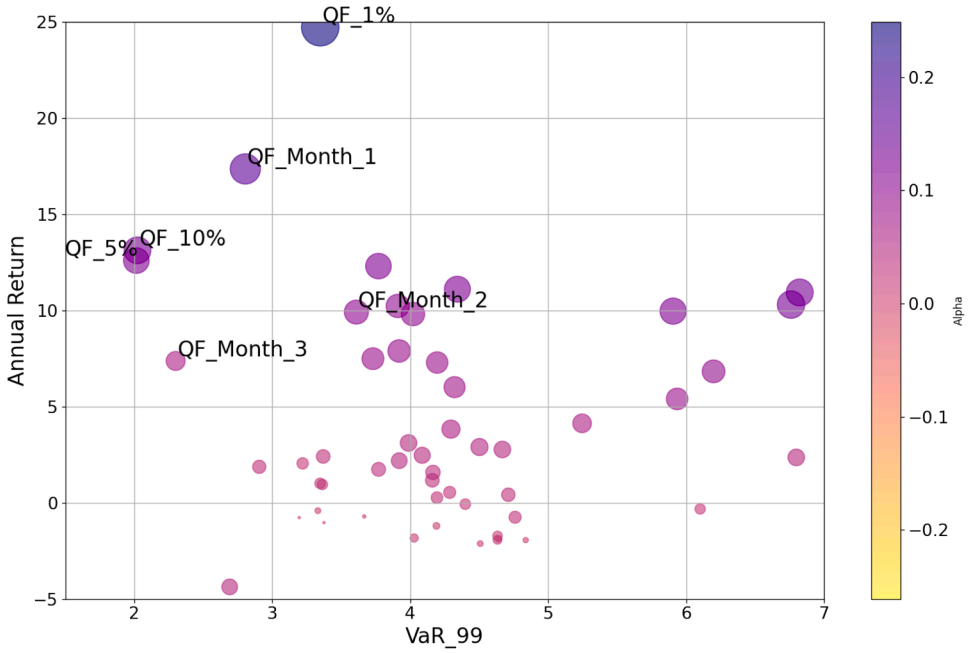


Fig. B.1. Comparison between factor performance and quantformer performance.

Appendix C. Symbol table

Symbol	Description
$A_{\text{peak}}, A_{\text{trough}}$	peak value of the portfolio
α	alpha ratio
$\mathbf{b}$	decision factor in the strategy
θ	bias in the model
β_p	beta ratio
D_p	daily return
h, H	attention head
χ_n^t, $\tilde{\chi}_n^t$	input feature matrix, normalized sequence
$\hat{Y}_n^t$	predicted output
$\mathbb{E}[x]$	mean
$\mathcal{G}^t$	set of the next timestamp profit
MD	max drawdown
t	timestep
m	the number of timesteps before t
MSELoss	MSE loss
n	the index of the stock
p^t	the close price of the stock
P	amount value of portfolio
Φ	trading strategy
Ψ	the empirical quantile CDF
R_f, R_m, R_p	return of risk-free rate, market return, return of portfolio
r_n^t	profit rate
s_n^t	stock
S^t	stock set
SR	Sharpe ratio
std$[x]$	standard deviation
TR	turnover rate of the portfolio
v_n^t	turnover rate of the stock over the time period t
VaR_α	value at risk at α level
w_n^t	weight of portfolio
x_n^t, $\tilde{x}_n^t$	row vector, normalized vector
y_n^t, $\tilde{y}_n^t$	target label, sorted predicted output
ϱ	number of bins in y_n^t
φ	percentage of stocks for each ϱ
ξ	middle percentage of y_n^t
$\mathcal{L}$	loss of holding stock
$\mathbf{F}$	multi-head attention output
$\mathbf{K}$, $\mathbf{Q}$, $\mathbf{V}$	key, query, value in quantformer
$\mathbf{W}$	trainable weight matrix
$\mathbf{Z}$	output layer

References

Araci, D. (2019). FinBERT: Financial sentiment analysis with pre-trained language models. arXiv:1908.10063.

Asness, C. S. (1995). The power of past stock returns to explain future stock returns. Goldman Sachs Asset Management, New York, NY.

Bacon, C. R. (2023). Practical portfolio performance measurement and attribution. Chennai: John Wiley & Sons.

Ban, G.-Y., El Karoui, N., & Lim, A. E. B. (2018). Machine learning and portfolio optimization. *Management Science*, *64*(3), 1136–1154.

Bao, W., Yue, J., & Rao, Y. (2017). A deep learning framework for financial time series using stacked autoencoders and long-short term memory. *PloS One*, *12*(7), e0180944.

Barez, F., Bilokon, P., Gervais, A., & Lisitsyn, N. (2023). Exploring the advantages of transformers for high-frequency trading. arXiv:2302.13850.

Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, *13*(2), 281–305.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., & Amodei, D. (2020). Language models are few-shot learners. In *Proceedings of the 34th international conference on neural information processing systems* NIPS'20

Cavalcante, R. C., Brasileiro, R. C., Souza, V. L. F., Nobrega, J. P., & Oliveira, A. L. I. (2016). Computational intelligence and financial markets: A survey and future directions. *Expert Systems with Applications*, *55*, 194–211.

Chen, W., Wang, W., Peng, B., Wen, Q., Zhou, T., & Sun, L. (2022). Learning to rotate: Quaternion transformer for complicated periodical time series forecasting. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 146–156).

Chen, Y., Zhao, H., Li, Z., & Lu, J. (2020). A dynamic analysis of the relationship between investor sentiment and stock market realized volatility: Evidence from China. *PloS One*, *15*(12), e0243080.

China securities regulatory commission (2002). Notice on adjusting the standards for securities transaction commissions. https://neris.csrc.gov.cn/falvfagui/rdqsHeader/mainbody?navbarId=1&secFutrsLawId=b25a17fb262742f2aea99515380cf3bb

Chowdhury, R. R., Zhang, X., Shang, J., Gupta, R. K., & Hong, D. (2022). Tarnet: Task-aware reconstruction for time-series transformer. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 212–220).

Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. In *Nips 2014 workshop on deep learning*.

Cirstea, R.-G., Guo, C., Yang, B., Kieu, T., Dong, X., & Pan, S. (2022). Triformer: Triangular, variable-specific attentions for long sequence multivariate time series forecasting. In *Proceedings of the thirty-first international joint conference on artificial intelligence (IJCAI-22)* (pp. 1994–2001).

De Prado, M. L. (2018). Advances in financial machine learning. New York: John Wiley & Sons.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the north american chapter of the association for computational linguistics: human language technologies, volume 1* (pp. 4171–4186).
Diamond, P. A. (1971). A model of price adjustment. *Journal of Economic Theory*, *3*(2), 156–168.
Ding, Q., Wu, S., Sun, H., Guo, J., & Guo, J. (2020). Hierarchical multi-scale Gaussian transformer for stock movement prediction. In *Ijcai* (pp. 4640–4646).
Ding, W., Mazouz, K., Ap Gwilym, O., & Wang, Q. (2023). Technical analysis as a sentiment barometer and the cross-section of stock returns. *Quantitative Finance*, *23*(11), 1617–1636.
Eisenach, C., Patel, Y., & Madeka, D. (2020). MQTransformer: Multi-horizon forecasts with context dependent and feedback-aware attention. arXiv:2009.14799.
Fama, E. F., & French, K. R. (1993). Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, *33*(1), 3–56.
Fama, E. F., & French, K. R. (2015). A five-factor asset pricing model. *Journal of Financial Economics*, *116*(1), 1–22.
Feng, F., He, X., Wang, X., Luo, C., Liu, Y., & Chua, T.-S. (2019). Temporal relational ranking for stock prediction. *ACM Transactions on Information Systems (TOIS)*, *37*(2), 1–30.
Fischer, T., & Krauss, C. (2018a). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, *270*(2), 654–669. ISSN 0377–2217
Fischer, T., & Krauss, C. (2018b). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, *270*(2), 654–669.
Fischer, T., Sterling, M., & Lessmann, S. (2024). Fx-spot predictions with state-of-the-art transformer and time embeddings. *Expert Systems with Applications*, *249*, 123538.
Gorishniy, Y., Rubachev, I., & Babenko, A. (2022). On embeddings for numerical features in tabular deep learning. *Advances in Neural Information Processing Systems*, *35*, 24991–25004.
Haritha, P. H., & Rishad, A. (2020). An empirical examination of investor sentiment and stock market volatility: Evidence from India. *Financial Innovation*, *6*(1), 1–15.
Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780.
Huang, W., Nakamori, Y., & Wang, S.-Y. (2005). Forecasting stock market movement direction with support vector machine. *Computers and Operations Research*, *32*(10), 2513–2522.
Huck, N. (2009). Pairs selection and outranking: An application to the S&P 100 index. *European Journal of Operational Research*, *196*(2), 819–825.
Hull, J. (2012). Risk management and financial institutions (vol. 733). Hoboken: John Wiley & Sons.
Kehinde, T. O., Adedokun, O. J., Joseph, A., Kabirat, K. M., Akano, H. A., & Olanrewaju, O. A. (2025a). Helformer: An attention-based deep learning model for cryptocurrency price forecasting. *Journal of Big Data*, *12*(1), 81.
Kehinde, T. O., Adedokun, O. J., Kareem, M. K., Akpan, J., & Olanrewaju, O. A. (2025b). STL-ELM: A computationally efficient hybrid approach for predicting high volatility stock market. *Intelligent Systems with Applications*, *27*, 200564.
Kehinde, T. O., Chan, F. T. S., & Chung, S. H. (2023). Scientometric review and analysis of recent approaches to stock market forecasting: Two decades survey. *Expert Systems with Applications*, *213*, 119299.
Kercheval, A. N., & Zhang, Y. (2015). Modelling high-frequency limit order book dynamics with support vector machines. *Quantitative Finance*, *15*(8), 1315–1329.
Kim, K.-j. (2003). Financial time series forecasting using support vector machines. *Neurocomputing*, *55*(1–2), 307–319.
Kim, S., Yun, S.-B., Bae, H.-O., Lee, M., & Hong, Y. (2024). Physics-informed convolutional transformer for predicting volatility surface. *Quantitative Finance*, *24*(2), 203–220.
Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In Y. Bengio, & Y. LeCun (Eds.), *3rd international conference on learning representations, ICLR 2015, san diego, ca, usa, may 7–9, 2015, conference track proceedings*.
Kitaev, N., Kaiser, L., & Levskaya, A. (2020). Reformer: The efficient transformer. In *In 8th international conference on learning representations (ICLR 2020)*.
Klambauer, G., Unterthiner, T., Mayr, A., & Hochreiter, S. (2017). Self-normalizing neural networks. *Advances in Neural Information Processing Systems*, *30*, 971–980.
Li, S., Jin, X., Xuan, Y., Zhou, X., Chen, W., Wang, Y.-X., & Yan, X. (2019). Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. *Advances in Neural Information Processing Systems*, *32*.
Liang, Y., Xia, Y., Ke, S., Wang, Y., Wen, Q., Zhang, J., Zheng, Y., & Zimmermann, R. (2023). Airformer: Predicting nationwide air quality in China with transformers. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 14329–14337). (*vol. 37*).
Lin, Y., Koprinska, I., & Rana, M. (2021). SSDNet: State space decomposition neural network for time series forecasting. In *2021 IEEE International conference on data mining (ICDM)* (pp. 370–378). IEEE.
Liu, M., Ren, S., Ma, S., Jiao, J., Chen, Y., Wang, Z., & Song, W. (2021). Gated transformer networks for multivariate time series classification. arXiv:2103.14438.
Liu, S., Yu, H., Liao, C., Li, J., Lin, W., Liu, A. X., & Dustdar, S. (2022a). Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting. In *In 10th international conference on learning representations (ICLR 2022)*.
Liu, Y., Hu, T., Zhang, H., Wu, H., Wang, S., Ma, L., & Long, M. (2024). Itransformer: Inverted transformers are effective for time series forecasting. In *The twelfth international conference on learning representations (ICLR 2024)*.
Liu, Y., Wu, H., Wang, J., & Long, M. (2022b). Non-stationary transformers: Exploring the stationarity in time series forecasting. *Advances in Neural Information Processing Systems*, *35*, 9881–9893.
Ma, B., Xue, Y., Lu, Y., & Chen, J. (2025). Stockformer: A price-volume factor stock selection model based on wavelet transform and multi-task self-attention networks. *Expert Systems with Applications*, *273*, 126803.
Markowitz, H. (1952). Portfolio selection. *Journal of Finance*, *7*(1), 77–91.
Moreno-Pino, F., Arroyo, Á., Waldon, H., Dong, X., & Cartea, Á. (2024). Rough transformers: Lightweight and continuous time series modelling through signature patching. In *Proceedings of the 38th international conference on neural information processing systems* (pp. 106264–106294).
Naseem, S., Mohsin, M., Hui, W., Liyan, G., & Penglai, K. (2021). The investor psychology and stock market behavior during the initial era of COVID-19: A study of China, Japan, and the United States. *Frontiers in Psychology*, *12*, 626934.
Nayak, R. K., Mishra, D., & Rath, A. K. (2015). A naïve SVM-KNN based stock market trend reversal analysis for Indian benchmark indices. *Applied Soft Computing*, *35*, 670–680.
Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2022). A time series is worth 64 words: Long-term forecasting with transformers. In *The eleventh international conference on learning representations (ICLR 2022)*.
Olorunnimbe, K., & Viktor, H. (2024). Ensemble of temporal transformers for financial time series. *Journal of Intelligent Information Systems*, *62*, 1087–1111.
Orsel, O. E., & Yamada, S. S. (2022). Comparative study of machine learning models for stock price prediction. arXiv:2202.03156.
O'Shea, K., & Nash, R. (2015). An introduction to convolutional neural networks. arXiv:1511.08458.
Pak, U., Kim, C., Ryu, U., Sok, K., & Pak, S. (2018). A hybrid model based on convolutional neural networks and long short-term memory for ozone concentration prediction. *Air Quality, Atmosphere and Health*, *11*, 883–895.
Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L. et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, *32*, 8026–8037.
Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I. et al. (2019). Language models are unsupervised multitask learners. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
Ramos-Pérez, E., Alonso-González, P. J., & Núñez-Velázquez, J. J. (2021). Multi-transformer: A new neural network-based architecture for forecasting S&P volatility. *Mathematics*, *9*(15), 1794.
Sezer, O. B., & Ozbayoglu, A. M. (2018). Algorithmic financial trading with deep convolutional neural networks: Time series to image conversion approach. *Applied Soft Computing*, *70*, 525–538.
Shabani, A., Abdi, A., Meng, L., & Sylvain, T. (2022). Scaleformer: Iterative multi-scale refining transformers for time series forecasting. arXiv:2206.04038.
Shanghai stock exchange (2023). Fee schedule of the Shanghai stock exchange. https://big5.sse.com.cn/site/cht/www.sse.com.cn/services/tradingservice/charge/ssecharge/.
Sharpe, W. (1964). Capital asset prices: A theory of market equilibrium under conditions of risk. *Journal of Finance*, *19*, 425–442.
Shenzhen stock exchange (2023). Fee schedule of the Shenzhen stock exchange. https://www.szse.cn/www/marketServices/deal/payFees/.
Shi, Y., Kong, L., Yang, L., Li, D., & Cui, X. (2024). Dynamic mean-variance portfolio selection under factor models. *Journal of Economic Dynamics and Control*, *167*, 104923.
So, D., Le, Q., & Liang, C. (2019). The evolved transformer. In *International conference on machine learning* (pp. 5877–5886). PMLR.
Sun, C., Wu, Q., & Yan, X. (2024). Dynamic CVaR portfolio construction with attention-powered generative factor learning. *Journal of Economic Dynamics and Control*, *160*, 104821.
Tang, B., & Matteson, D. S. (2021). Probabilistic transformer for time series analysis. *Advances in Neural Information Processing Systems*, *34*, 23592–23608.
Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Proceedings of the 31st international conference on neural information processing systems* NIPS'17 (p. 6000-6010).
Wang, C., Chen, Y., Zhang, S., & Zhang, Q. (2022a). Stock market index prediction using deep transformer model. *Expert Systems with Applications*, *208*, 118128.
Wang, H., Wang, T., Li, S., Zheng, J., Guan, S., & Chen, W. (2022b). Adaptive long-short pattern transformer for stock investment selection. In *Proceedings of the thirty-first international joint conference on artificial intelligence* (pp. 3970–3977).
Wang, S. (2023). A stock price prediction method based on BiLSTM and improved transformer. *IEEE Access*, *11*, 104211–104223.
Wei, J., Xu, Q., & He, C. (2022). Deep learning of predicting closing price through historical adjustment closing price. *Procedia Computer Science*, *202*, 379–384.
Wu, C., Wu, F., & Huang, Y. (2021a). DA-Transformer: Distance-aware transformer. In *Proceedings of the 2021 conference of the North American chapter of the association for computational linguistics: human language technologies* (pp. 2059–2068).

Wu, H., Xu, J., Wang, J., & Long, M. (2021b). Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in Neural Information Processing Systems*, *34*, 22419–22430.
Wu, S., Irsoy, O., Lu, S., Dabravolski, V., Dredze, M., Gehrmann, S., Kambadur, P., Rosenberg, D., & Mann, G. (2023). BloombergGPT: A large language model for finance. arXiv:2303.17564.
Wu, S., Xiao, X., Ding, Q., Zhao, P., Wei, Y., & Huang, J. (2020). Adversarial sparse transformer for time series forecasting. *Advances in Neural Information Processing Systems*, *33*, 17105–17115.
Xu, K., Zhang, Y., Ye, D., Zhao, P., & Tan, M. (2021). Relation-aware transformer for portfolio policy learning. In *Proceedings of the twenty-ninth international conference on international joint conferences on artificial intelligence* (pp. 4647–4653).
Yang, Y., Uy, M., & Huang, A. (2020). FinBERT: A pretrained language model for financial communications. arXiv:2006.08097.
Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023a). Are transformers effective for time series forecasting? In *Proceedings of the AAAI conference on artificial intelligence* (pp. 11121–11128). (*vol. 37*).
Zeng, Z., Kaur, R., Siddagangappa, S., Rahimi, S., Balch, T., & Veloso, M. (2023b). Financial time series forecasting using CNN and transformer. arXiv:2304.04912.
Zerveas, G., Jayaraman, S., Patel, D., Bhamidipaty, A., & Eickhoff, C. (2021). A transformer-based framework for multivariate time series representation learning. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining* (pp. 2114–2124).
Zhang, L., Aggarwal, C., & Qi, G.-J. (2017). Stock price prediction via discovering multi-frequency trading patterns. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 2141–2149).
Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021). Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 11106–11115). (*vol. 35*).
Zhou, T., Ma, Z., Wen, Q., Wang, X., Sun, L., & Jin, R. (2022). Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International conference on machine learning* (pp. 27268–27286). PMLR.
Zhu, S., & Fukushima, M. (2009). Worst-case conditional value-at-risk with application to robust portfolio management. *Operations Research*, *57*(5), 1155–1168.
Zuo, S., Jiang, H., Li, Z., Zhao, T., & Zha, H. (2020). Transformer Hawkes process. In *International conference on machine learning* (pp. 11692–11702). PMLR.